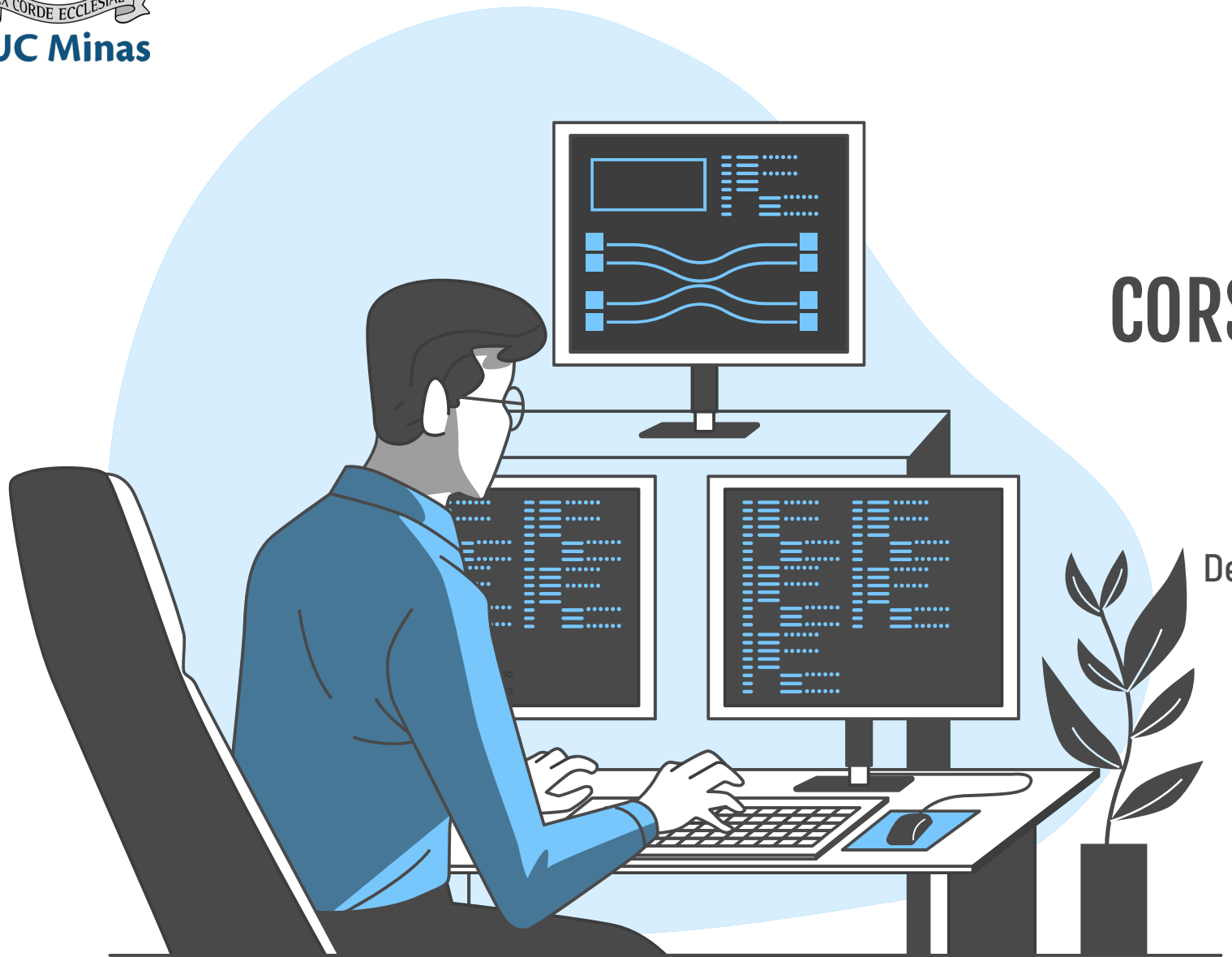
PUC Minas



Aula 03 · FullStack

CORS: liberando o front para chamar o back

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1

Entender a **política de mesma origem** e o que é **CORS**

2

Reconhecer o erro de CORS no **Console** e na aba **Network**

3

Saber o que é a requisição **preflight** (`OPTIONS`)

4

Configurar CORS com `WebMvcConfigurer`, `@CrossOrigin` e no Spring Security

5

Ler as origens permitidas do `application.properties`

Antes de começar

- Aulas 01 e 02 desta pasta

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 O que acontece

02 Preflight: o OPTIONS antes do POST

03 Solução 1 (recomendada): configuração global

04 Solução 2: @CrossOrigin no controller

05 Solução 3: com Spring Security no projeto

06 Checklist quando aparecer erro de CORS

07 Aprofundando: CORS em detalhes

01

O que acontece

O que acontece

Por segurança, o navegador **não deixa** um JavaScript carregado de uma origem (`http://localhost:5173`) ler respostas de outra origem (`http://localhost:8080`), a não ser que o **servidor** diga que permite. Esse "dizer que permite" é o **CORS** (*Cross-Origin Resource Sharing*): cabeçalhos HTTP como `Access-Control-Allow-Origin` na resposta.

01

O que acontece

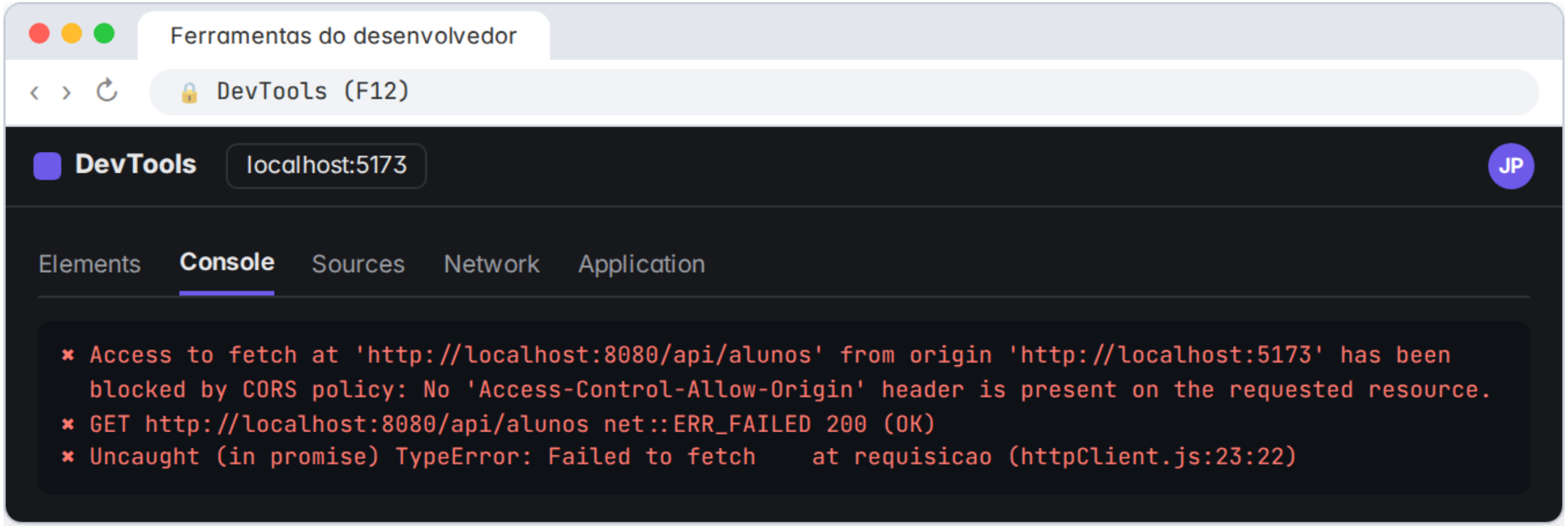
Aula 03 · FullStack

O que acontece

Observação

CORS é uma regra **do navegador**. Por isso a mesma requisição funciona no cliente HTTP, no `curl` e no Postman, e falha só no React. Não é bug do seu código Java: é o Spring não ter autorizado a origem do front.

O que acontece



Erro de CORS no Console. Repare: o servidor até respondeu **200**, mas o navegador escondeu a resposta do JavaScript.

O que acontece

Do lado do React, o erro chega como `TypeError: Failed to fetch`, o mesmo de quando o back está desligado. Por isso o `httpClient.js` dos projetos mostra a mensagem *"Confira se o Spring Boot está rodando e se o CORS libera este endereço"*.

01

O que acontece

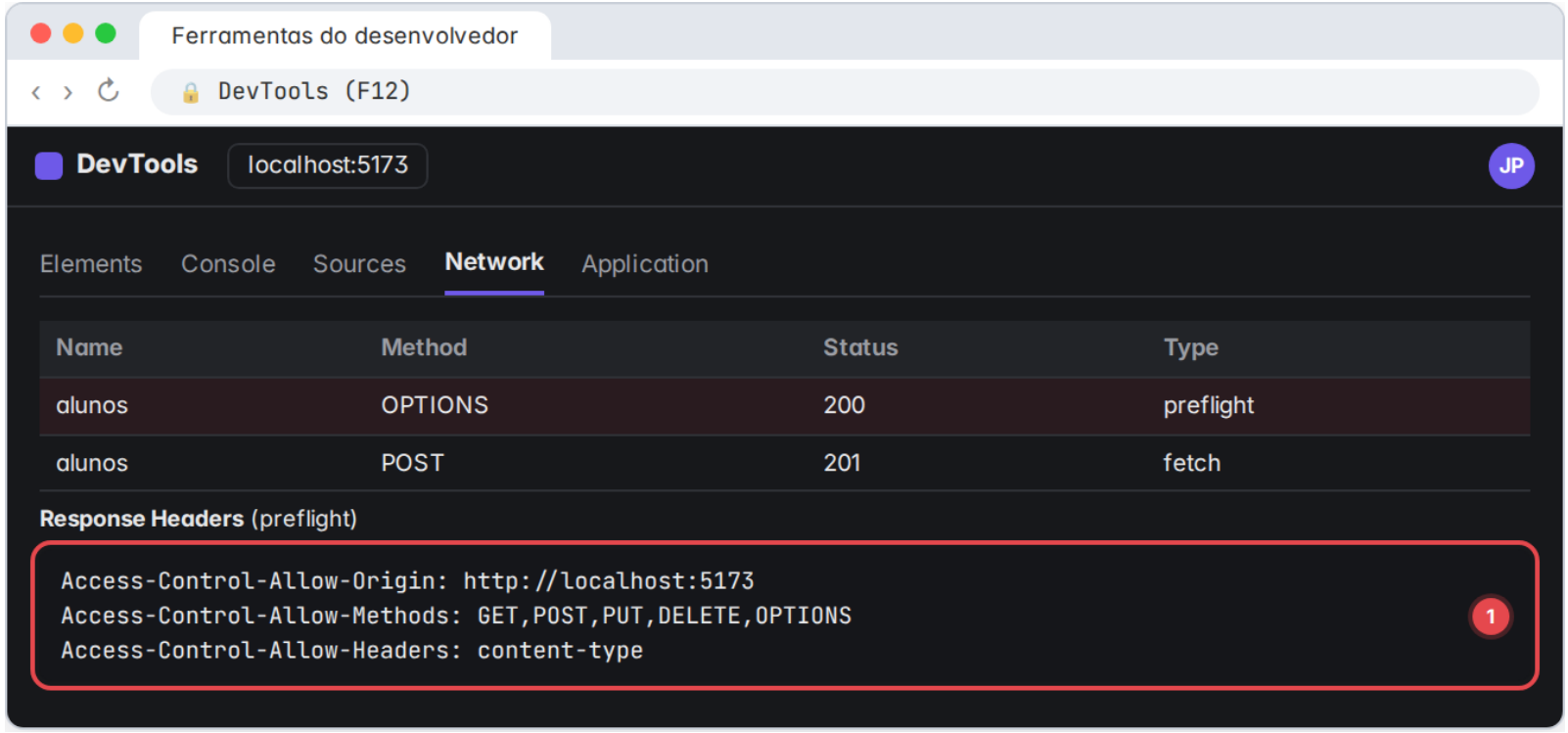
Aula 03 · FullStack

02

Preflight: o OPTIONS antes do POST

Preflight: o OPTIONS antes do POST

Para requisições “não simples” (com `Content-Type: application/json`, cabeçalho `Authorization`, ou métodos `PUT`/`DELETE`), o navegador primeiro manda um **`OPTIONS`** perguntando se pode. Só se a resposta autorizar ele envia a requisição de verdade.

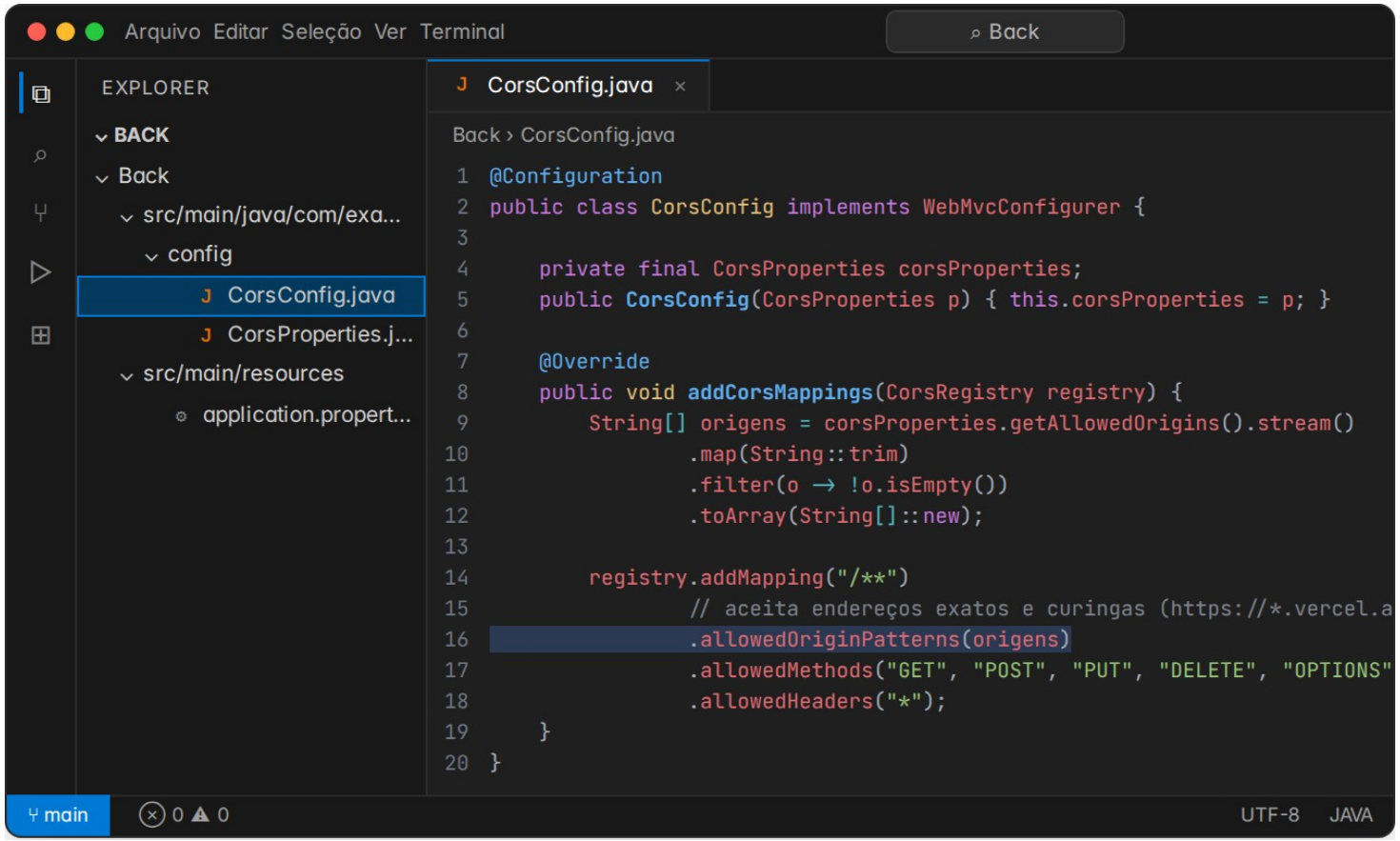


Aba **Network** com o *preflight* `OPTIONS` autorizado e o `POST` em seguida.

03

Solução 1 (recomendada): configuração global

Solução 1 (recomendada): configuração global



É a usada em todos os projetos `FullStack`. Uma classe no pacote `config` libera as origens lidas do `application.properties`:

`CorsConfig` do `Exemplo_Integracao`: as origens vêm de `app.cors.allowed-origins`.

Solução 1 (recomendada): configuração global

application.properties

```
# local (npm run dev) + Vercel. Separe por vírgula e NÃO coloque "/" no final.
app.cors.allowed-origins=${CORS_ORIGINS:http://localhost:5173,https://exemplo-integracao-front.vercel.app}
```

Solução 1 (recomendada): configuração global

Dica

O curinga `https://*.vercel.app` (aceito por `allowedOriginPatterns`) libera também as URLs de *preview* que o Vercel cria para cada branch, como `meu-app-git-feature-joao.vercel.app`. Em produção séria, prefira listar só o domínio oficial.

04

Solução 2: @CrossOrigin no controller

Solução 2: @CrossOrigin no controller

Rápida para testes, mas espalha a configuração pelos controllers:

```
@CrossOrigin
```

```
@CrossOrigin(origins = "http://localhost:5173")
@RestController
@RequestMapping("/api/alunos")
public class AlunoController { ... }
```

05

Solução 3: com Spring Security no projeto

Solução 3: com Spring Security no projeto

Os filtros de segurança rodam **antes** do Spring MVC. Se o projeto tem Spring Security, o *preflight* `OPTIONS` é barrado antes de chegar ao `CorsConfig`. É preciso ligar o CORS **dentro** da `SecurityFilterChain` e fornecer um `CorsConfigurationSource` (veja Back-end, aula 09):

CORS + Spring Security

```
http.cors(Customizer.withDefaults())   // procura um bean CorsConfigurationSource
    .csrf(csrf -> csrf.disable())
    ...

@Bean
CorsConfigurationSource corsConfigurationSource(CorsProperties props) {
    var cfg = new CorsConfiguration();
    cfg.setAllowedOriginPatterns(props.getAllowedOrigins());
    cfg.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
    cfg.setAllowedHeaders(List.of("*"));          // inclui Authorization (JWT)
    var src = new UrlBasedCorsConfigurationSource();
    src.registerCorsConfiguration("/**", cfg);
    return src;
}
```

06

Checklist quando aparecer erro de CORS

Checklist quando aparecer erro de CORS

Sintoma	Causa provável	Correção
`No 'Access-Control-Allow-Origin' header`	Origem não liberada	Adicione a origem exata em `app.cors.allowed-origins`
Funcionava local, quebrou no Vercel	Faltou a URL do Vercel	Cadastre `CORS_ORIGINS` no Render com a URL do Vercel e redeploy
Origem liberada e mesmo assim falha	Barra no final: `https://app.vercel.app/`	A origem **nunca** tem `/` no final
`http` x `https`	Liberou `http://app.vercel.app`	O Vercel usa `https`
Só `POST`/`DELETE` falham	Método não está em `allowedMethods`	Inclua os métodos e `OPTIONS`
401/403 no `OPTIONS`	Spring Security sem `.cors(...)`	Solução 3
`Failed to fetch` sem mensagem de CORS	Back desligado ou URL errada	Confira `VITE_API_URL` e se o Spring está no ar

Checklist quando aparecer erro de CORS

Atenção

Evite `allowedOrigins("*")` como “solução”. Além de liberar qualquer site, ele **não funciona** junto com `allowCredentials(true)` (cookies). Liste as origens.

07

Aprofundando: CORS em detalhes

Aprofundando: CORS em detalhes

Cabeçalho da resposta	Significado
`Access-Control-Allow-Origin`	Qual origem pode ler a resposta
`Access-Control-Allow-Methods`	Métodos permitidos (no preflight)
`Access-Control-Allow-Headers`	Cabeçalhos permitidos, como `Authorization`
`Access-Control-Allow-Credentials`	Se cookies podem ser enviados
`Access-Control-Max-Age`	Por quantos segundos o navegador guarda o preflight

Requisição simples x com preflight

Simples (sem OPTIONS)

- GET/HEAD/POST
- Cabeçalhos básicos
- `Content-Type` de formulário ou texto

vs

Com preflight

- PUT, DELETE, PATCH
- `Content-Type: application/json`
- Cabeçalho `Authorization` (JWT)

Aprofundando: CORS em detalhes

CorsConfig mais restrito

```
registry.addMapping("/api/**")
        .allowedOriginPatterns(origens)
        .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
        .allowedHeaders("Content-Type", "Authorization")
        .exposedHeaders("Location")          // deixa o front ler o cabeçalho Location
        .maxAge(3600);                        // 1 h sem repetir o preflight
```

Aprofundando: CORS em detalhes

Simulando o preflight com curl

```
$ curl -i -X OPTIONS http://localhost:8080/api/alunos \
    -H "Origin: http://localhost:5173" \
    -H "Access-Control-Request-Method: POST" \
    -H "Access-Control-Request-Headers: content-type"

HTTP/1.1 200
Access-Control-Allow-Origin: http://localhost:5173
Access-Control-Allow-Methods: GET,POST,PUT,DELETE,OPTIONS
Access-Control-Allow-Headers: content-type
Access-Control-Max-Age: 1800
```

Não é CORS (embora pareça)

Back desligado

`Failed to fetch` sem mensagem de CORS no console.

Mixed content

Site `https` chamando API `http`: o navegador bloqueia antes.

Erro 500 sem cabeçalho

Uma exceção antes do filtro de CORS pode gerar resposta sem `Allow-Origin`.

Extensões do navegador

Bloqueadores de anúncio às vezes barram chamadas para `/api/track`...

Aprofundando: CORS em detalhes

Observação

Uma resposta **500** costuma aparecer no console como erro de CORS, porque a página de erro padrão pode sair sem os cabeçalhos. Antes de mexer no CORS, olhe o **log do Spring**: se houver uma exceção, corrija-a primeiro.

08

Quiz de revisão

Quiz · questão 1 de 5

Pergunta

CORS é uma regra de quem?

A Do Spring

B Do navegador

C Do banco

D Do Vercel

Quiz · questão 1 de 5

Resposta

CORS é uma regra de quem?

A Do Spring

✓ Do navegador

C Do banco

D Do Vercel

Por quê: Por isso curl e Insomnia não sofrem CORS.

Pergunta

O que é o preflight?

A Um teste de carga

B Uma requisição OPTIONS que o navegador envia antes

C O build do Vite

D Um tipo de token

Quiz · questão 2 de 5

Resposta

O que é o preflight?

A Um teste de carga

✓ Uma requisição OPTIONS que o navegador envia antes

C O build do Vite

D Um tipo de token

Por quê: Disparado por JSON, Authorization, PUT ou DELETE.

Pergunta

Qual origem está escrita corretamente para o CORS?

A https://app.vercel.app/

B https://app.vercel.app

C app.vercel.app/

D http://app.vercel.app/

Resposta

Qual origem está escrita corretamente para o CORS?

A https://app.vercel.app/

✓ https://app.vercel.app

C app.vercel.app/

D http://app.vercel.app/

Por quê: Sem barra final e com https.

Pergunta

Com Spring Security, onde ligar o CORS?

A Só no WebMvcConfigurer

B Na SecurityFilterChain com `.cors(...)`

C No front

D No pom.xml

Resposta

Com Spring Security, onde ligar o CORS?

A — Só no WebMvcConfigurer

✓ — Na SecurityFilterChain com `.cors(...)`

C — No front

D — No pom.xml

Por quê: Os filtros de segurança rodam antes do MVC.

Pergunta

No React, um erro de CORS chega como:

A 404

B TypeError: Failed to fetch

C 500

D Nenhum erro

Resposta

No React, um erro de CORS chega como:

A 404

✓ TypeError: Failed to fetch

C 500

D Nenhum erro

Por quê: Igual ao back desligado; confira o console.

Fechamento

Resumo da aula

- ✓ O navegador bloqueia leituras entre origens diferentes; o **servidor** autoriza com cabeçalhos CORS.
- ✓ O erro aparece no Console; o front recebe `Failed to fetch`; cliente HTTP e curl não sofrem CORS.
- ✓ Requisições com JSON, `Authorization`, `PUT` ou `DELETE` disparam um *preflight* `OPTIONS`.
- ✓ Configuração global (`CorsConfig` + `CorsProperties`) com origens no `application.properties`.
- ✓ Com Spring Security, ligue `.cors(...)` na `SecurityFilterChain`.

Exercícios

1. Remova a origem `localhost:5173` do `application.properties`, reproduza o erro e capture o print do Console.

2. Libere novamente usando a variável de ambiente `CORS_ORIGINS` em vez de editar o arquivo.

3. Na aba Network, encontre um *preflight* e anote os cabeçalhos da resposta.

4. Adicione Spring Security ao projeto e faça o CORS continuar funcionando.

Para ir além

- developer.mozilla.org/pt-BR/docs/Web/HTTP/CORS
- docs.spring.io/spring-framework/reference/web/webmvc-cors.html

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 03?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web